SQL Server Modern Query Processing

Gianluca Hotz

ghotz@ugiss.org | @glhotz

SQL START! 2021 – 10° ANNIVERSARY

Chi sono?

- Gianluca Hotz | @glhotz | ghotz@ugiss.org
- Consulente indipendente
  - 25 anni su SQL Server (dalla 4.21 nel 1996)
  - Modellazione e sviluppo database, dimensionamento e amministrazione database server, aggiornamenti e migrazioni, performance tuning
- Community
  - 23 anni Microsoft MVP SQL Server/Data Platform (dal 1998)
  - VMware Experts SQL Server
  - Fondatore e presidente UGISS (~~PASS Chapter~~)
  - Co-organizzatore DAMAG Meetup Community

Partners

Dev>Marche

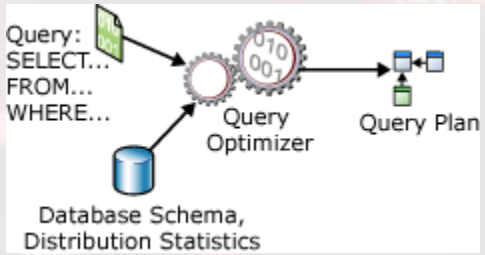
UNIVERSITÀ POLITECNICA DELLE MARCHE

UNISS

Agenda

- Introduzione
- Cardinality Estimator (2014+)
- Auto Tuning (2017+)
- Intelligent Query Processing (2017+, 2019+)

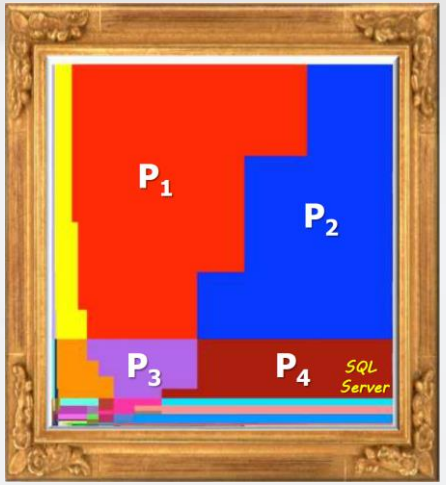
«Query Optimizer»



- SQL è un linguaggio dichiarativo
  - Specifichiamo cosa vogliamo ottenere, non come
- Il QO genera un piano di esecuzione
  - Il come ottenere le cose richieste
  - Tipicamente cercandolo tra molteplici piani possibili
  - Sfruttando modello basato su costi e statistiche distribuzione valori
  - https://docs.microsoft.com/en-gb/sql/relational-databases/query-processing-architecture-guide

Un po’ di storia

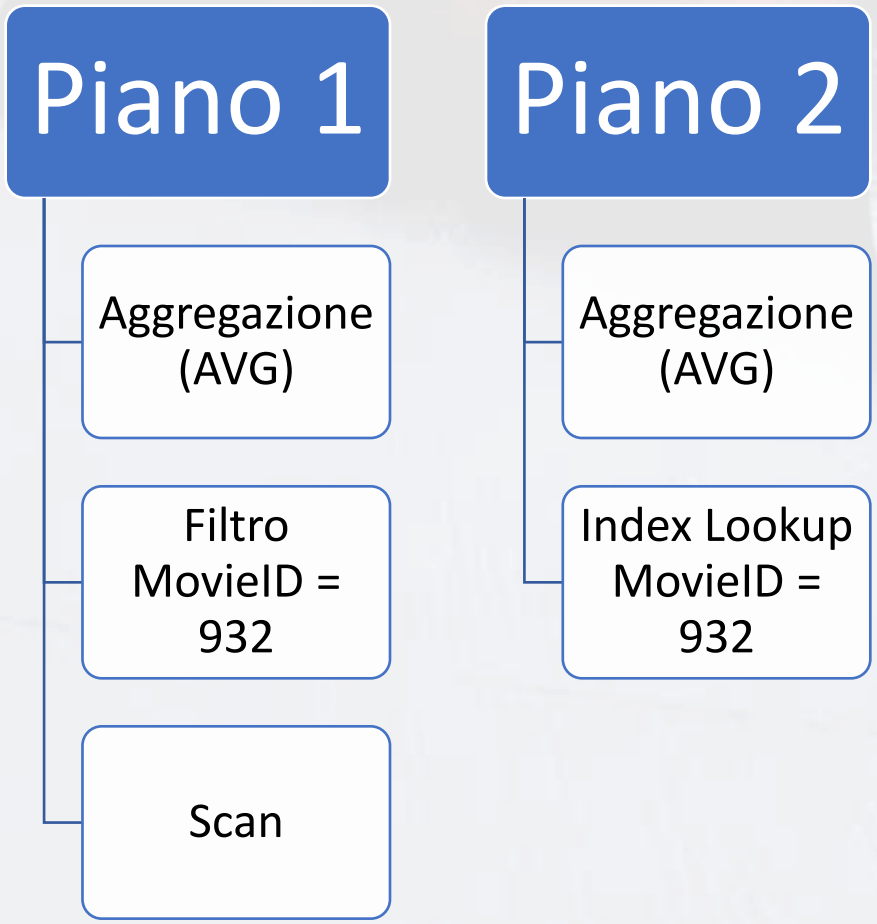
- Ottimizzazione «cost-based» risale 1970
  - Pat Selinger in System R (poi divenuto DB2)
- 50 anni, RDBMS ancora software più difficile da implementare
  - Paura di causare regressioni
- Obiettivo QO non miglior piano in assoluto…
  - Spazio delle soluzioni troppo grande per essere esplorato
  - Miglior piano in un tempo ragionevole (approccio euristico)
  - Sceglie ancora troppi piani inefficienti ☹



Esempio

- SELECT AVG(Rating)
  FROM Reviews
  WHERE MovieID = 932
- Calcolare media punteggi recensioni per film con ID = 932

Esempio

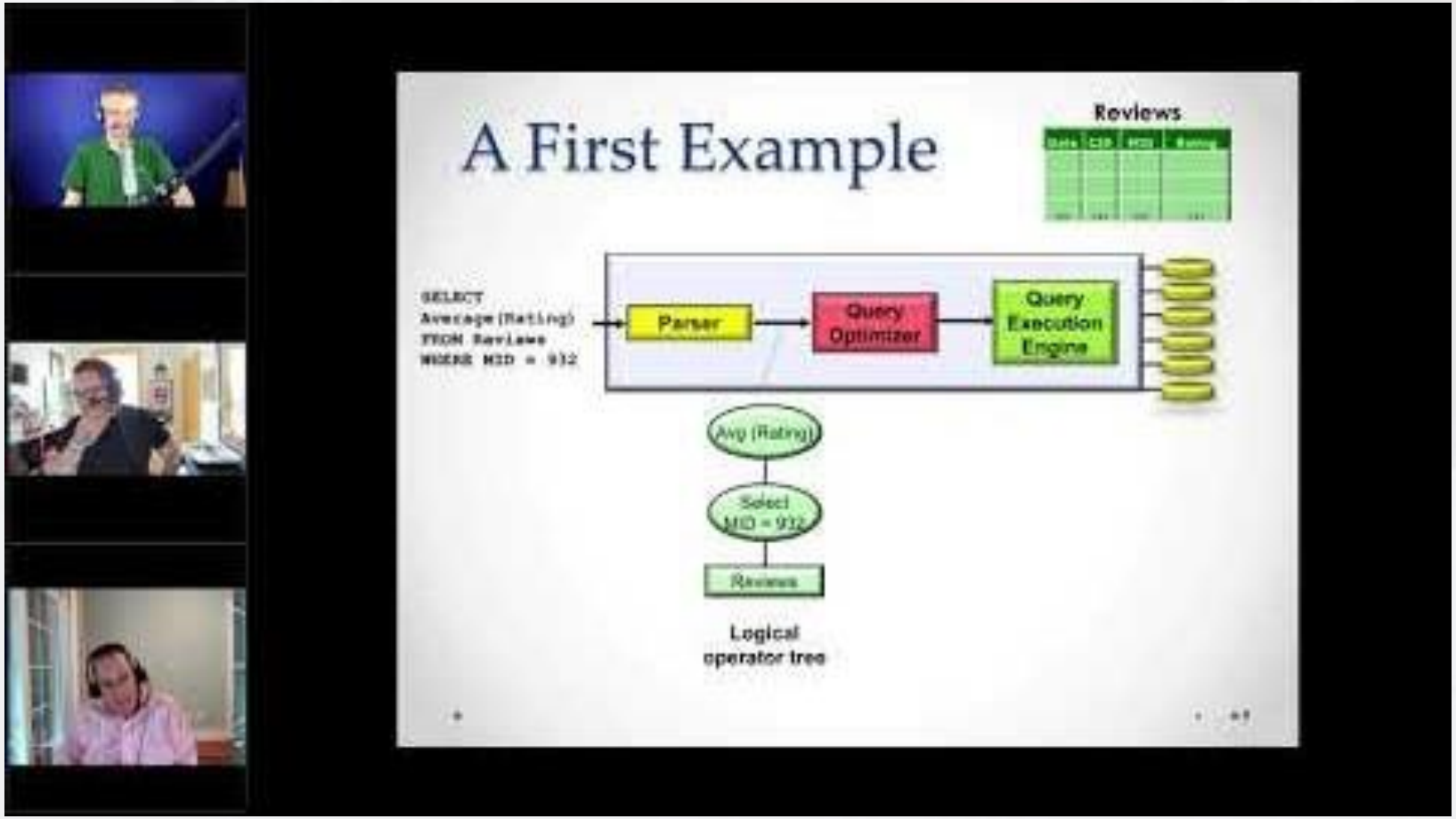

Piano 1
Aggregazione (AVG)
Filtro MovieID = 932
Scan
Piano 2
Aggregazione (AVG)
Index Lookup MovieID = 932

Esempio più complesso

- TPC_H Query 8
  - National Market Share
- Circa 22 milioni di piani possibili!

```
Select o_year,
sum(case
 when nation = 'BRAZIL' then volume
 else 0
end) / sum(volume)
from
(
 select YEAR(O_ORDERDATE) as o_year,
  L_EXTENDEDPRICE * (1 - L_DISCOUNT) as volume,
  n2.N_NAME as nation
 from PART, SUPPLIER, LINEITEM, ORDERS, CUSTOMER, NATION
n1,
 NATION n2, REGION
 where
    P_PARTKEY = L_PARTKEY and S_SUPPKEY = L_SUPPKEY
    and L_ORDERKEY = O_ORDERKEY and O_CUSTKEY = C_CUSTKEY
    and C_NATIONKEY = n1.N_NATIONKEY and n1.N_REGIONKEY =
R_REGIONKEY
    and R_NAME = 'AMERICA‘ and S_NATIONKEY = n2.N_NATIONKEY
    and O_ORDERDATE between '1995-01-01' and '1996-12-31'
    and P_TYPE = 'ECONOMY ANODIZED STEEL'
    and S_ACCTBAL <= constant-1
    and L_EXTENDEDPRICE <= constant-2
) as all_nations
group by o_year  order by o_year
```

SQL Query Optimization: Why Is It So Hard to Get Right?



Nuovo «Cardinality Estimator»

SQL Server Modern Query Processing

Introduzione al «Cardinality Estimator»

- Componente del «Query Processor»
- Essenziale per generare piani esecuzione, predice
  - Numero righe risultati intermedi (es. join, filtri, aggregazioni)
  - Numero righe finale
- In gran parte basato su SQL Server 7.0
  - Abilitazione Fix in QFE tramite Trace Flag per evitare bug di regressione
  - Alcuni problemi necessitavano di un ridisegno rilevante
- SQL Server 2014+ nuovo CE può causare regressioni
  - La maggior parte delle query dovrebbe comunque beneficiarne

Demo

- Cardinality Estimator

Risorse nuovo CE

- Articoli
  - Miloš Radivojević Blog Series
    - http://milossql.wordpress.com/tag/cardinality-estimator
  - A First Look at the New SQL Server Cardinality Estimator
    - http://www.sqlperformance.com/2013/12/t-sql-queries/a-first-look-at-the-new-sql-server-cardinality-estimator
  - New functionality in SQL Server 2014 – Part 2 – New Cardinality Estimation
    - http://blogs.msdn.com/b/saponsqlserver/archive/2014/01/16/new-functionality-in-sql-server-2014-part-2-new-cardinality-estimation.aspx
  - Join Containment Assumption and CE Model Variation in SQL Server
    - https://www.sqlshack.com/join-containment-assumption-and-ce-model-variation

«Automatic Tuning»

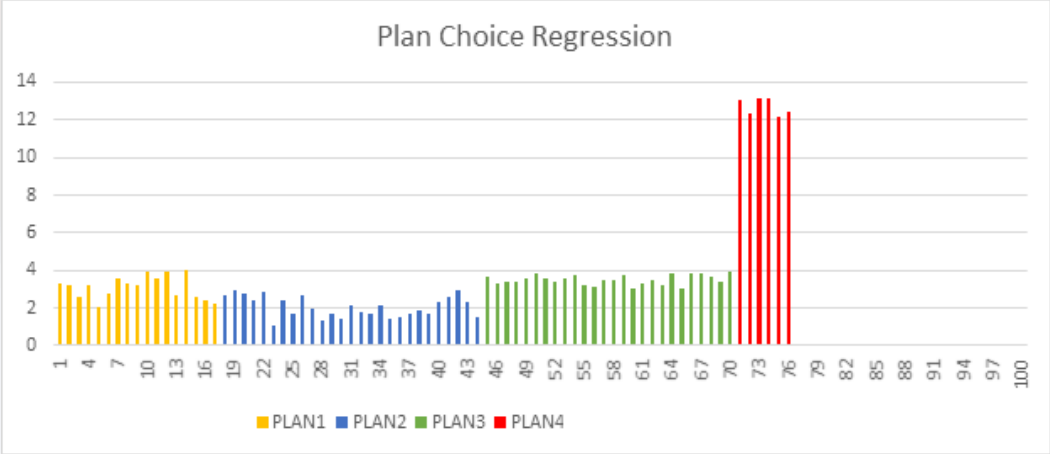
SQL Server Modern Query Processing

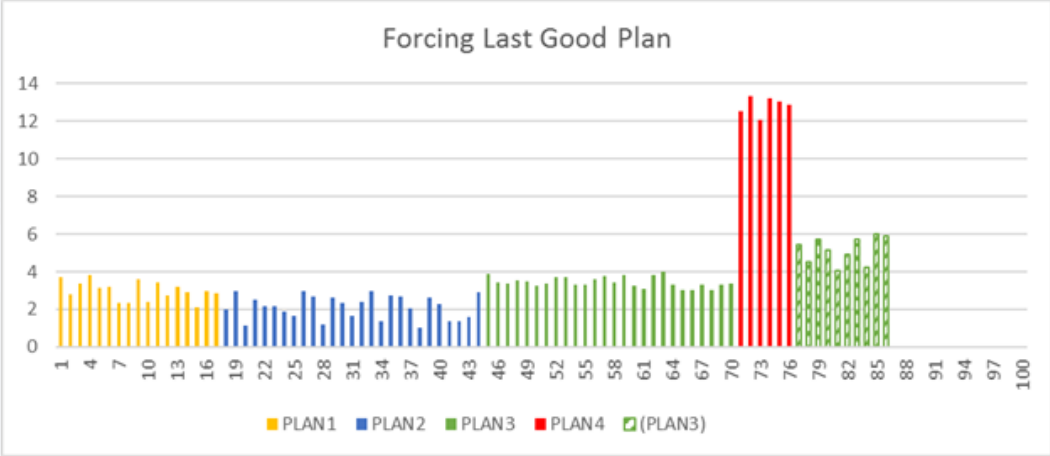
«Tuning» automatico

- Correzione dei piani automatica
  - Forza ultimo piano ottimale quando identifica regressione
  - Disponibile in SQL Server 2017+ EE e Azure SQL Database
- Gestione automatica indici
  - Creazione/eliminazione indici
  - Verifica effettivo utilizzo e miglioramento (modello ML)
  - Disponibile solo in Azure SQL Database

Correzione automatica piani

- Forza ultimo piano ottimale
  - quando identifica regressione
  - contro storico Query Store
  - non sopravvive al ravvio
- Miglioramento minimo
  - di 10 secondi «CPU time»
- Solo Enterprise Edition
- Correzione manuale
  - 2016+ report Query Store
  - 2017+ nuova DMV
    - **sys.dm_db_tuning_recommendations**





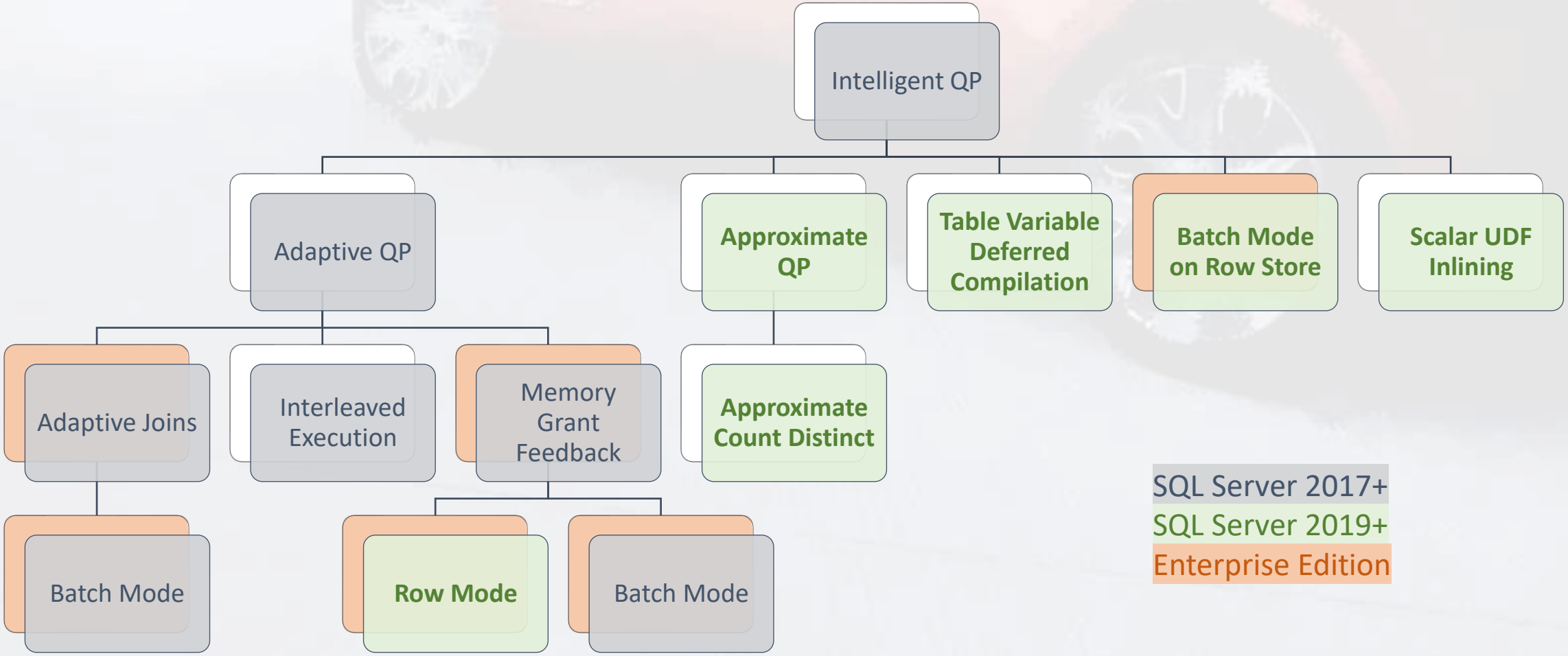
Demo

- Correzione dei piani automatica

«Intelligent Query Processing»

SQL Server Modern Query Processing

Intelligent Query Processing



SQL Server 2017+
SQL Server 2019+
Enterprise Edition

«Execution Modes»

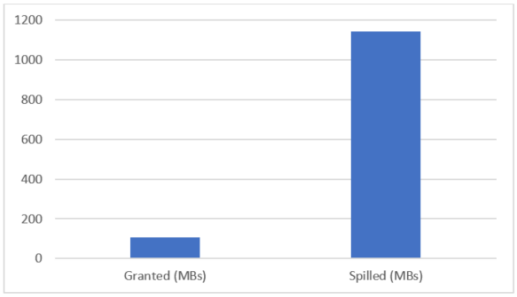
- «Row Mode»
  - Iteratori albero piano esecuzione consumano 1 riga alla volta
  - Modalità esecuzione tradizionale con «Rowstore»
- «Batch Mode»
  - Iteratori albero piano esecuzione consumano batch di righe alla volta (1000)
  - Ottimale per operazioni di «scan» (es. aggregati o join tra tabelle grosse)
  - SQL Server 2012: introdotto per sfruttare al meglio indici Columnstore
  - SQL Server 2016/2017: estesi scenari di utilizzo con indici Columnstore
  - SQL Server 2019: estesi scenari di utilizzo con «Rowstore»

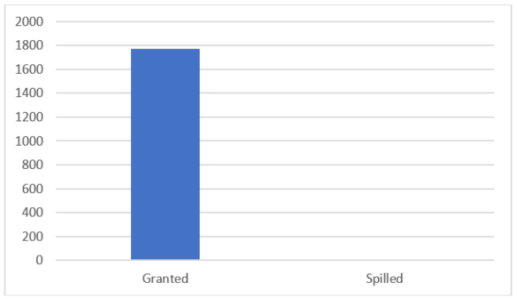
Demo

- «Batch Mode» con «Rowstore»

Memory Grant Feedback

- Valutazione post-esecuzione
  - Aggiorna memoria «granted» per i piani in cache
  - Es. più memoria se «spill», meno se «grant» eccesivo
- Versioni supportate
  - SQL Server 2017+ «Batch Mode»
  - SQL Server 2019+ «Row Mode»
- «Plan caching»
  - Non persistente (non salvato nel Query Store)
  - OPTION(RECOMPILE) previene «caching» e «memory grant feedback»



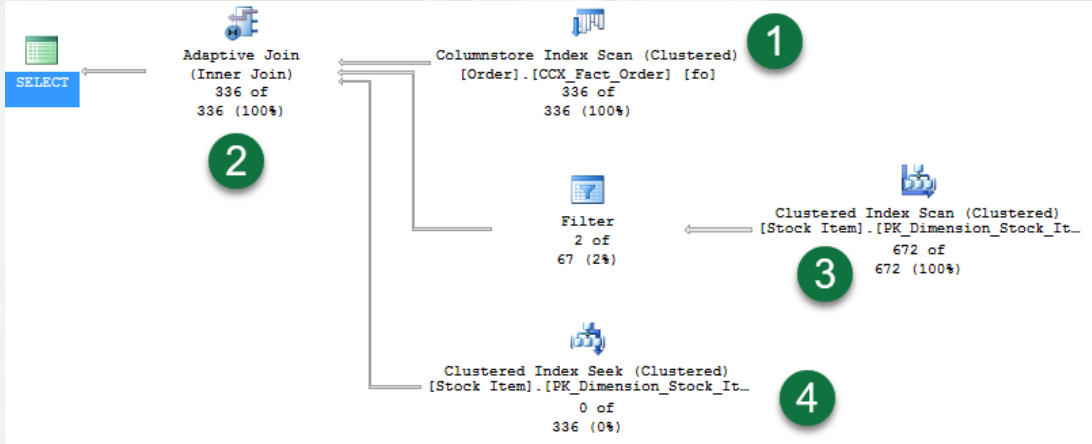


Demo

- «Memory Grant Feedback»

«Batch Mode Adaptive Joins»

- «Nested Loop Join» efficiente
  - tabelle input molto piccole
  - una tabella input piccola + indice per «seek» su tabella grande
- «Hash Join» efficiente
  - tabelle di input molto voluminose
  - potenzialmente molta memoria
- «Adaptive Joins»
  - rinvia la scelta dopo scansione primo input



Demo

- «Batch Mode Adaptive Joins»

«Interleaved Execution»

- Problema con funzioni che ritornano tabelle
  - «multi-statement table valued functions» (MSTVF)
  - SQL Server <= 2012 QO assume cardinalità = 1
  - SQL Server 2014/2016 QO assuma cardinalità = 100
- SQL Server >= 2017
  - Inizia ottimizzazione
  - Esegue MSTVF se candidata per «Interleaved Execution»
  - Riprende ottimizzazione con la cardinalità corretta

Demo

- «Interleaved Execution»

Compilazione differita variabili tabella

- Compilazione comando che referenzia variabile tabella
  - SQL Server < 2019 compilato prima che sia popolata con cardinalità fissa = 1
  - SQL Server 2019+ differito a prima esecuzione (come tabelle temporanee)

Demo

- Compilazione differita variabili tabella

«Scalar UDF In-lining»

- Funzioni utente T-SQL (UDF) che ritornano un singolo valore scalare
- Problemi di prestazioni
  - Invocazione iterativa linguaggio interpretato
    - Per ogni riga, «context switching» molto oneroso durante esecuzione query
    - Comandi eseguiti in isolamento, no ottimizzazioni «cross-statement
  - Esecuzione seriale
    - Parallelismo «Intra-query» inibito
  - Mancanza di costi
    - Storicamente attribuiti solo a operatori relazioni, assunzione costo UDF basso…

«Scalar UDF Automatic In-lining»

- SQL Server 2019+ UDF scalari automaticamente convertite in
  - Espressioni scalari
  - «Subquery» scalari
- Ottimizzazione piano completa (UDF non più visibili)

IQP: esempio «Scalar UDF in-lining»

```
CREATE FUNCTION dbo.discount_price(@price DECIMAL(12,2), @discount DECIMAL(12,2))
RETURNS DECIMAL (12,2) AS BEGIN RETURN @price * (1 - @discount); END

SELECT L_SHIPDATE, O_SHIPPRIORITY
, SUM(dbo.discount_price(L_EXTENDEDPRICE, L_DISCOUNT))
FROM LINEITEM, ORDERS
WHERE O_ORDERKEY = L_ORDERKEY
GROUP BY L_SHIPDATE, O_SHIPPRIORITY
ORDER BY L_SHIPDATE
```

10GB CCI compressed TPC-H Schema, 2 x CPUs (12 cores), 96GB RAM, SSD storage

	Query senza UDF	Query con UDF (senza in-lining)	Query con UDF (e in-lining)
Tempo esecuzione	1.6 secondi	29 minuti 11 secondi	1.6 secondi

Demo

- «Scalar UDF Automatic In-lining»

APPROX_COUNT_DISTINCT

- Ritorna il numero **approssimato** di valori univoci non nulli per gruppo
  - Algoritmo «HyperLogLog» garantisce tasso errore <= 2% nel 97% dei casi
- Risultati rapidi con consumo di memoria ridotto
  - Es. dashboards, analisi di trend, «feature selection», ecc.
  - Pensato per grossi volumi di dati e richieste
    - es. 10 miliardi di righe, 1 utente che consuma 1,5GB di memoria per risultato preciso contro 100 utenti in contemporanea che usano 12MB per un risultato approssimato…
- Compromesso
  - precisione, quindi <u>solo scenari dove i valori precisi non sono necessari</u>!

Demo

- APPROX_COUNT_DISTINCT

Disponibilità per edizione

Standard, Web, Express

- New Cardinality Estimator
- Interleaved Execution
- Table variable deferred compilation
- Scalar UDF In-lining
- Approximate Distinct Count

Enterprise, Developer

- Automatic Tuning
- Batch Mode on Rowstore
- Memory Grant Feedback
- Adaptive Joins

Grazie! Domande?

SQL Server in AWS